# Dynamic Programming – String Problems

Longest Palindromic Subsequence, Word Break Problem, Word Wrap Problem

**GUIDE:** Dr. SRIRAMYA
**NAME:** JAASIEL J.S

## 1. Longest Palindromic Subsequence

### Question 1: DNA Palindrome Motif Detection

A bioinformatics tool scans a DNA string over {A,C,G,T} to find the length of the longest palindromic subsequence, which hints at regions that could fold back on themselves. Only the length is required, not the exact subsequence.

**Approach**

Reverse the string and compute the Longest Common Subsequence (LCS) between the original and its reverse, since the longest palindromic subsequence of s equals LCS(s, reverse(s)). Fill a 2D DP table bottom-up using the standard LCS recurrence and read the answer from the last cell.

**Complexity Analysis**

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Every cell of the n×n table is computed once from its neighboring cells. |
| Space | $O(n^2)$ | The full table is kept so a later question can reconstruct the actual subsequence. |

**Test Cases**

```
assert lps_length("bbbab") == 4
assert lps_length("cbbd") == 2
assert lps_length("") == 0
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    r = s[::-1]
    n = len(s)
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            if s[i - 1] == r[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][n]

print("LPS length of bbbab:", lps_length("bbbab"))
print("LPS length of cbbd :", lps_length("cbbd"))
assert lps_length("bbbab") == 4
assert lps_length("cbbd") == 2
assert lps_length("") == 0
print("All test cases passed!")
```

Output

```
LPS length of bbbab: 4
LPS length of cbbd : 2
All test cases passed!
```

### Question 2: Palindrome-Based Word Game Scorer

A word game scores each submitted string by the length of its longest palindromic subsequence — higher scores reward more hidden symmetry. Implement the scorer and use it to rank two submitted words.

### Approach

Use the same LPS DP (dp[i][j] = LPS length of substring i..j, filling by increasing substring length, extending by matching outer characters or taking the max of dropping either end). Compute the score for each submission and compare.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The DP table for each submitted word is filled once, per word. |
| Space | O(n) | Only the score is needed, so the table is compressed to two rolling rows instead of the full grid. |

### Test Cases

```
assert palindrome_score("bbbab") > palindrome_score("cbbd")
assert palindrome_score("a") == 1
print('All test cases passed!')
```

main.py

```
def palindrome_score(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            if s[i] == s[j]:
                dp[j] = prev + 2
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev = old
    return dp[-1] if n else 0

a, b = "bbbab", "cbbd"
print(a, "score =", palindrome_score(a))
print(b, "score =", palindrome_score(b))
print("Winner:", a if palindrome_score(a) > palindrome_score(b) else b)
assert palindrome_score(a) > palindrome_score(b)
assert palindrome_score("a") == 1
print("All test cases passed!")
```

Output

```
bbbab score = 4
cbbd score = 2
Winner: bbbab
All test cases passed!
```

Question 3: Chat Message Symmetry Score

A chat moderation tool flags unusually symmetric messages (a possible sign of bot spam) by measuring what fraction of a message's characters form a palindromic subsequence. Implement the ratio computation.

### Approach

Compute the LPS length with the standard DP, then divide by the message length to get a symmetry ratio between 0 and 1, flagging messages whose ratio exceeds a threshold.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The standard LPS recurrence is applied once per message. |
| Space | $O(n^2)$ | The naive table is kept, though a banded implementation could reduce memory for very long messages. |

### Test Cases

```
assert round(symmetry_ratio("bbbab"), 2) == 0.8
assert round(symmetry_ratio("cbbd"), 2) == 0.5
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = 2 + (dp[i + 1][j - 1] if j > i + 1 else 0)
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp[0][n - 1] if n else 0

def symmetry_ratio(message):
    return lps_length(message) / len(message) if message else 0.0

print("bbbab ratio =", round(symmetry_ratio("bbbab"), 2))
print("cbbd ratio  =", round(symmetry_ratio("cbbd"), 2))
assert round(symmetry_ratio("bbbab"), 2) == 0.8
assert round(symmetry_ratio("cbbd"), 2) == 0.5
print("All test cases passed!")
```

Output

```
bbbab ratio = 0.8
cbbd ratio  = 0.5
All test cases passed!
```

Question 4: RNA Secondary-Structure Base-Pairing Estimate

An RNA folding tool estimates the maximum number of complementary base pairs that could form within a single strand by modeling it as a longest palindromic subsequence problem over a simplified alphabet.

### Approach

Apply the LPS recurrence directly, treating the fragment as an ordinary string; since only the count of base pairs matters, the reported pair count is half the LPS length, rounded down.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The DP table is filled once for the fragment before halving the final value. |
| Space | O(n) | A rolling-row compression is used since the fragment can be long and only the final count is needed. |

### Test Cases

```
assert base_pair_estimate("bbbab") == 2
assert base_pair_estimate("cbbd") == 1
print('All test cases passed!')
```

main.py

```
def lps_length_optimized(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            if s[i] == s[j]:
                dp[j] = prev + 2
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev = old
    return dp[-1] if n else 0

def base_pair_estimate(fragment):
    return lps_length_optimized(fragment) // 2

print("bbbab base pairs =", base_pair_estimate("bbbab"))
print("cbbd base pairs  =", base_pair_estimate("cbbd"))
assert base_pair_estimate("bbbab") == 2
assert base_pair_estimate("cbbd") == 1
print("All test cases passed!")
```

Output

```
bbbab base pairs = 2
cbbd base pairs  = 1
All test cases passed!
```

Question 5: Data De-duplication Redundancy Score

A storage system estimates how compressible a string is by checking how much of it participates in a palindromic subsequence — higher overlap suggests redundant structure worth compressing.

### Approach

Reuse the LPS DP table; the redundancy score is defined as LPS length minus half the string length, and a positive score marks the string as a compression candidate.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The full LPS DP is computed once per candidate string. |
| Space | O(n) | Only the score (not the reconstructed subsequence) is required, so rows are compressed. |

### Test Cases

```
assert is_compression_candidate("bbbab") == True   # 4 - 2.5 > 0
assert is_compression_candidate("cbbd") == False   # 2 - 2 == 0
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            if s[i] == s[j]:
                dp[j] = prev + 2
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev = old
    return dp[-1] if n else 0

def is_compression_candidate(s):
    redundancy = lps_length(s) - len(s) / 2
    return redundancy > 0

print("bbbab candidate =", is_compression_candidate("bbbab"))
print("cbbd candidate  =", is_compression_candidate("cbbd"))
assert is_compression_candidate("bbbab") is True
assert is_compression_candidate("cbbd") is False
print("All test cases passed!")
```

Output

```
bbbab candidate = True
cbbd candidate  = False
All test cases passed!
```

Question 6: Reconstructing the Actual Subsequence for a Puzzle App

A puzzle app doesn't just need the length of the longest palindromic subsequence — it needs to highlight the actual characters to the player. Implement reconstruction by backtracking through the DP table.

### Approach

Build the full LPS DP table, then backtrack from dp[0][n-1]: if the outer characters match, include both and move inward; otherwise move toward whichever neighboring cell (dropping the left or right character) produced the larger value.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Filling the table takes $O(n^2)$; the backtracking walk adds only O(n). |
| Space | $O(n^2)$ | The full table must be retained for the backtracking pass, unlike length-only variants. |

### Test Cases

```
assert reconstruct_lps("bbbab") == "bbbb"
assert reconstruct_lps("cbbd") == "bb"
print('All test cases passed!')
```

main.py

```
def reconstruct_lps(s):
    n = len(s)
    if n == 0:
        return ""
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = 2 + (dp[i + 1][j - 1] if j > i + 1 else 0)
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

    left, right = [], []
    i, j = 0, n - 1
    while i <= j:
        if i == j:
            left.append(s[i])
            break
        if s[i] == s[j] and dp[i][j] == 2 + (dp[i + 1][j - 1] if j > i + 1 else 0):
            left.append(s[i]); right.append(s[j])
            i += 1; j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    return "".join(left + right[::-1])

print("LPS of bbbab =", reconstruct_lps("bbbab"))
print("LPS of cbbd  =", reconstruct_lps("cbbd"))
assert reconstruct_lps("bbbab") == "bbbb"
assert reconstruct_lps("cbbd") == "bb"
print("All test cases passed!")
```

Output

```
LPS of bbbab = bbbb
LPS of cbbd  = bb
All test cases passed!
```

Question 7: Memory-Optimized LPS for Long Genome Strings

A genome-analysis pipeline needs the LPS length for strings that can be hundreds of thousands of characters long, where an $O(n^2)$ memory table would exhaust available RAM. Implement a length-only version using O(n) space.

### Approach

Since each row of the DP table only depends on the row below it and that row shifted by one, keep just two rolling arrays (previous and current row) instead of the full 2D table, discarding earlier rows as the computation progresses.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The number of cell computations is unchanged from the full-table version. |
| Space | O(n) | Memory drops from quadratic to linear since only two rows are ever live at once. |

### Test Cases

```
assert lps_length_optimized("bbbab") == 4
assert lps_length_optimized("cbbd") == 2
print('All test cases passed!')
```

main.py

```
def lps_length_optimized(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev_diag = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            if s[i] == s[j]:
                dp[j] = prev_diag + 2
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev_diag = old
    return dp[-1] if n else 0

print("bbbab ->", lps_length_optimized("bbbab"))
print("cbbd  ->", lps_length_optimized("cbbd"))
assert lps_length_optimized("bbbab") == 4
assert lps_length_optimized("cbbd") == 2
print("All test cases passed!")
```

Output

```
bbbab -> 4
cbbd  -> 2
All test cases passed!
```

Question 8: Password Weakness Checker

A password-strength auditor flags passwords that contain an unusually long palindromic subsequence, since such structure can make a password easier to guess or remember (and thus weaker).

### Approach

Compute the LPS length with the standard DP and compare it against a configurable threshold relative to the password's total length; passwords exceeding the threshold are flagged for review.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Passwords are short, so this cost is negligible in practice. |
| Space | O(n) | The rolling-row optimization is used since only the pass/fail flag is needed. |

### Test Cases

```
assert is_weak_password("bbbab", threshold=0.6) == True
assert is_weak_password("cbbd", threshold=0.6) == False
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            if s[i] == s[j]:
                dp[j] = prev + 2
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev = old
    return dp[-1] if n else 0

def is_weak_password(password, threshold=0.6):
    if not password:
        return False
    return lps_length(password) / len(password) > threshold

print("bbbab weak =", is_weak_password("bbbab", 0.6))
print("cbbd weak  =", is_weak_password("cbbd", 0.6))
assert is_weak_password("bbbab", threshold=0.6) is True
assert is_weak_password("cbbd", threshold=0.6) is False
print("All test cases passed!")
```

Output

```
bbbab weak = True
cbbd weak  = False
All test cases passed!
```

Question 9: Log Sequence Symmetry Auditing

A monitoring system audits sequences of event codes for suspicious symmetric patterns (a sign of replayed or spoofed traffic) by computing the longest palindromic subsequence of each log window.

### Approach

Slide a fixed-size window over the log stream and recompute the LPS length for each window using the standard DP; since windows are small and fixed-size, the recomputation cost per window stays bounded.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n \cdot w^2)$ | Each of n windows of size w costs $O(w^2)$, for a total of $O(n \cdot w^2)$ over the stream. |
| Space | O(w) | A rolling-row compression is used per window. |

### Test Cases

```
assert lps_length("bbbab") == 4  # example window flagged as symmetric
assert lps_length("cbbd") == 2   # example window not flagged
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    n = len(s)
    dp = [0] * n
    for i in range(n - 1, -1, -1):
        prev = 0
        dp[i] = 1
        for j in range(i + 1, n):
            old = dp[j]
            dp[j] = prev + 2 if s[i] == s[j] else max(dp[j], dp[j - 1])
            prev = old
    return dp[-1] if n else 0

def audit_log_windows(log_stream, window_size):
    return [(log_stream[i:i + window_size],
             lps_length(log_stream[i:i + window_size]))
            for i in range(len(log_stream) - window_size + 1)]

print("bbbab score =", lps_length("bbbab"))
print("cbbd score  =", lps_length("cbbd"))
print("Audit sample:", audit_log_windows("bbbab", 5))
assert lps_length("bbbab") == 4
assert lps_length("cbbd") == 2
print("All test cases passed!")
```

Output

```
bbbab score = 4
cbbd score  = 2
Audit sample: [('bbbab', 4)]
All test cases passed!
```

Question 10: Comparing LPS vs Longest Common Subsequence Reformulation

A course assignment asks students to prove and empirically verify that the longest palindromic subsequence of a string equals the longest common subsequence of the string and its reverse.

### Approach

Implement lps_length using the direct palindrome DP recurrence, and separately implement lcs_length(s, reverse(s)) using the standard two-string LCS DP, then compare the two results on the same input.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Each of the two independent DP computations costs $O(n^2)$ on its own. |
| Space | $O(n^2)$ | Both tables are kept simultaneously for comparison, or O(n) each if computed one at a time. |

### Test Cases

```
assert lps_length("bbbab") == lcs_length("bbbab", "bbbab"[::-1])
assert lps_length("cbbd") == lcs_length("cbbd", "cbbd"[::-1])
print('All test cases passed!')
```

main.py

```
def lps_length(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = 2 + (dp[i + 1][j - 1] if j > i + 1 else 0)
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp[0][n - 1] if n else 0

def lcs_length(a, b):
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            dp[i][j] = dp[i - 1][j - 1] + 1 if a[i - 1] == b[j - 1] else max(dp[i - 1][j], dp[i][j - 1])
    return dp[-1][-1]

for s in ("bbbab", "cbbd"):
    print(s, "LPS =", lps_length(s), "LCS(reverse) =", lcs_length(s, s[::-1]))
assert lps_length("bbbab") == lcs_length("bbbab", "bbbab"[::-1])
assert lps_length("cbbd") == lcs_length("cbbd", "cbbd"[::-1])
print("All test cases passed!")
```

Output

```
bbbab LPS = 4 LCS(reverse) = 4
cbbd LPS = 2 LCS(reverse) = 2
All test cases passed!
```

## 2. Word Break Problem

Question 1: Search Query Segmentation

A search engine receives a run-on query like "leetcode" (no spaces) and must decide if it can be segmented into words from a known dictionary before suggesting corrections, e.g. "leet code".

### Approach

Build a boolean DP array where dp[i] is true if the prefix s[0:i] can be fully segmented using dictionary words; for each i, check every valid split point j<i where dp[j] is true and s[j:i] is a dictionary word.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | There are n split points, each checked against dictionary words up to O(n) substring length. |
| Space | O(n) | Only the boolean DP array is kept, plus the dictionary's own hash-set storage. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("applepenapple", ["apple","pen"]) == True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')
```

main.py

```
def can_segment(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[-1]

print("leetcode      ->", can_segment("leetcode", ["leet", "code"]))
print("applepenapple ->", can_segment("applepenapple", ["apple", "pen"]))
print("catsandog     ->", can_segment("catsandog", ["cats", "dog", "sand", "and", "cat"]))
assert can_segment("leetcode", ["leet","code"]) is True
assert can_segment("applepenapple", ["apple","pen"]) is True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) is False
print("All test cases passed!")
```

Output

```
leetcode      -> True
applepenapple -> True
catsandog     -> False
All test cases passed!
```

Question 2: URL Slug Validator for an SEO Tool

An SEO tool checks whether a URL slug (with hyphens stripped) still reads as valid words from a site's keyword dictionary, to catch slugs that accidentally concatenate unrelated terms.

### Approach

Reuse the standard word-break DP on the stripped slug string against the keyword dictionary; a false result flags the slug for manual review before publishing.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The same split-point DP is run once per slug. |
| Space | O(n) | The DP array size is independent of dictionary size once the dictionary is stored in a hash set. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')
```

main.py

```
def can_segment(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        dp[i] = any(dp[j] and s[j:i] in word_set for j in range(i))
    return dp[-1]

def valid_slug(slug, keywords):
    return can_segment(slug.replace("-", ""), keywords)

print("leet-code valid  =", valid_slug("leet-code", ["leet", "code"]))
print("cats-and-og valid =", valid_slug("cats-and-og", ["cats", "dog", "sand", "and", "cat"]))
assert can_segment("leetcode", ["leet","code"]) is True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) is False
print("All test cases passed!")
```

Output

```
leet-code valid  = True
cats-and-og valid = False
All test cases passed!
```

Question 3: Spam Keyword-Stuffing Detector

A spam filter suspects keyword stuffing when a run-on token can be cleanly decomposed into several dictionary words back-to-back (e.g. "buycheapwatches"), since legitimate random tokens rarely segment this neatly.

### Approach

Run the word-break DP, but instead of a boolean result, count the minimum number of dictionary words needed for a valid segmentation; a low word count relative to token length raises the spam-suspicion score.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The same split-point structure is reused, tracking a minimum count instead of a boolean. |
| Space | O(n) | The DP array tracks, at each position, the minimum segment count achievable so far. |

### Test Cases

```
assert min_segments("applepenapple", ["apple","pen"]) == 3
assert min_segments("catsandog", ["cats","dog","sand","and","cat"]) is None
print('All test cases passed!')
```

main.py

```
def min_segments(s, words):
    word_set = set(words)
    inf = len(s) + 1
    dp = [inf] * (len(s) + 1)
    dp[0] = 0
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] != inf and s[j:i] in word_set:
                dp[i] = min(dp[i], dp[j] + 1)
    return None if dp[-1] == inf else dp[-1]

print("applepenapple ->", min_segments("applepenapple", ["apple", "pen"]))
print("catsandog     ->", min_segments("catsandog", ["cats", "dog", "sand", "and", "cat"]))
assert min_segments("applepenapple", ["apple","pen"]) == 3
assert min_segments("catsandog", ["cats","dog","sand","and","cat"]) is None
print("All test cases passed!")
```

Output

```
applepenapple -> 3
catsandog     -> None
All test cases passed!
```

Question 4: OCR Post-Processing for Scanned Text

An OCR pipeline sometimes drops spaces between words in scanned documents; a post-processing step checks whether the resulting run-on text is even segmentable before attempting reconstruction.

### Approach

Apply the word-break boolean DP against a large dictionary; if segmentable, a later step can recover the actual split, but this stage only needs the yes/no feasibility check.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The feasibility DP is run once over the recovered text block. |
| Space | O(n) | Feasibility only needs the boolean array, not a full path-reconstruction structure. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("applepenapple", ["apple","pen"]) == True
print('All test cases passed!')
```

main.py

```
def can_segment(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[-1]

def ocr_text_is_segmentable(text, dictionary):
    return can_segment(text.replace(" ", ""), dictionary)

print("leetcode segmentable      =", ocr_text_is_segmentable("leetcode", ["leet", "code"]))
print("applepenapple segmentable =", ocr_text_is_segmentable("applepenapple", ["apple", "pen"]))
assert can_segment("leetcode", ["leet","code"]) is True
assert can_segment("applepenapple", ["apple","pen"]) is True
print("All test cases passed!")
```

Output

```
leetcode segmentable      = True
applepenapple segmentable = True
All test cases passed!
```

Question 5: Chat Auto-Correct for No-Space Input

A messaging app's auto-correct feature detects when a user typed a valid multi-word phrase without spaces (common on some mobile keyboards) and offers to insert the spaces automatically.

### Approach

Run the word-break DP against the app's dictionary; if dp[n] is true, walk back through recorded split points to recover one valid spacing to suggest to the user.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The DP fill costs $O(n^2)$; the backtracking walk to recover a spacing adds O(n). |
| Space | O(n) | Both the DP array and the parent-pointer array used for reconstruction are linear in size. |

### Test Cases

```
assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
assert suggest_spacing("catsandog", ["cats","dog","sand","and","cat"]) is None
print('All test cases passed!')
```

main.py

```
def suggest_spacing(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    parent = [-1] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                parent[i] = j
                break
    if not dp[-1]:
        return None
    parts, i = [], len(s)
    while i:
        j = parent[i]
        parts.append(s[j:i])
        i = j
    return " ".join(reversed(parts))

print("leetcode  ->", suggest_spacing("leetcode", ["leet", "code"]))
print("catsandog ->", suggest_spacing("catsandog", ["cats", "dog", "sand", "and", "cat"]))
assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
assert suggest_spacing("catsandog", ["cats","dog","sand","and","cat"]) is None
print("All test cases passed!")
```

Output

```
leetcode  -> leet code
catsandog -> None
All test cases passed!
```

Question 6: Tokenizing Identifiers Against a Keyword Vocabulary

A code-analysis tool checks whether a poorly named identifier like "getuserdata" is actually a concatenation of recognizable vocabulary words, to suggest a properly spaced rename.

### Approach

Lower-case the identifier and run the standard word-break DP against a curated vocabulary of common programming terms, reporting both feasibility and one valid segmentation.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Identifiers are typically short, keeping the split-point search cheap in practice. |
| Space | O(n) | The DP and parent arrays are linear; the vocabulary is stored in a hash set for O(1) lookups. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
print('All test cases passed!')
```

main.py

```
def suggest_spacing(s, words):
    s = s.lower()
    word_set = {w.lower() for w in words}
    dp = [False] * (len(s) + 1)
    parent = [-1] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i], parent[i] = True, j
                break
    if not dp[-1]:
        return None
    parts, i = [], len(s)
    while i:
        j = parent[i]
        parts.append(s[j:i]); i = j
    return " ".join(reversed(parts))

print("getuserdata ->", suggest_spacing("getuserdata", ["get", "user", "data"]))
assert suggest_spacing("leetcode", ["leet","code"]) == "leet code"
print("All test cases passed!")
```

Output

```
getuserdata -> get user data
All test cases passed!
```

Question 7: All Possible Segmentations for Query Suggestions

A search box wants to show a user every plausible way a run-on query could be split into dictionary words, not just one — e.g. offering both "cat sand dog" and "cats and dog" as suggestions.

### Approach

Extend the word-break DP to Word Break II: first run the boolean DP to prune impossible prefixes, then use memoized recursion that, for each feasible prefix, branches over every valid word ending there and combines results from the remaining suffix.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ amortized | Memoization avoids recomputing shared suffixes; worst case grows with the number of valid segmentations. |
| Space | $O(n^2)$ | Memo storage of per-index results plus the accumulated output strings. |

### Test Cases

```
result = all_segmentations("catsanddog", ["cats","cat","and","sand","dog"])
assert "cats and dog" in result
assert "cat sand dog" in result
assert len(result) == 2
print('All test cases passed!')
```

main.py

```
from functools import lru_cache

def all_segmentations(s, words):
    word_set = set(words)

    @lru_cache(None)
    def solve(i):
        if i == len(s):
            return ("",)
        ans = []
        for j in range(i + 1, len(s) + 1):
            word = s[i:j]
            if word in word_set:
                for tail in solve(j):
                    ans.append(word if not tail else word + " " + tail)
        return tuple(ans)

    return list(solve(0))

result = all_segmentations("catsanddog", ["cats","cat","and","sand","dog"])
print("Segmentations:", result)
assert "cats and dog" in result
assert "cat sand dog" in result
assert len(result) == 2
print("All test cases passed!")
```

Output

```
Segmentations: ['cat sand dog', 'cats and dog']
All test cases passed!
```

Question 8: Pruning Impossible Prefixes Before Full Enumeration

Generating all segmentations directly can be wasteful when large portions of the string are not segmentable at all; a preprocessing step should skip full enumeration for strings that fail a simple boolean check first.

### Approach

Run the $O(n^2)$ boolean word-break DP as a cheap gate; only if the full string is feasible does the pipeline proceed to the more expensive all-segmentations enumeration from Question 7, avoiding wasted work on clearly invalid input.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The gate check avoids the enumeration entirely on infeasible input. |
| Space | $O(n)$ | The gate needs only the boolean array, versus $O(n^2)$+ for the full enumeration it may skip. |

### Test Cases

```
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) == False
assert can_segment("catsanddog", ["cats","cat","and","sand","dog"]) == True
print('All test cases passed!')
```

main.py

```
def can_segment(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        dp[i] = any(dp[j] and s[j:i] in word_set for j in range(i))
    return dp[-1]

def should_enumerate(s, words):
    return can_segment(s, words)

invalid = should_enumerate("catsandog", ["cats","dog","sand","and","cat"])
valid = should_enumerate("catsanddog", ["cats","cat","and","sand","dog"])
print("catsandog  -> enumerate?", invalid)
print("catsanddog -> enumerate?", valid)
assert invalid is False
assert valid is True
print("All test cases passed!")
```

Output

```
catsandog  -> enumerate? False
catsanddog -> enumerate? True
All test cases passed!
```

Question 9: Memoized Word Break for a Large Recurring Dictionary Stream

A streaming text pipeline repeatedly checks many overlapping substrings from a long text against the same dictionary; naive recomputation from scratch for each query wastes work.

### Approach

Cache DP results keyed by starting index across overlapping queries on the same underlying text, so previously computed feasibility for a given suffix is reused instead of recomputed for each new query window.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ first, O(1) after | The first full computation costs $O(n^2)$; subsequent cached lookups on the same text are amortized O(1). |
| Space | O(n) | The cached DP array is retained between queries on the same text. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("leetcode", ["leet","code"]) == True  # served from cache
print('All test cases passed!')
```

main.py

```
_cache = {}
cache_hits = 0

def can_segment(s, words):
    global cache_hits
    key = (s, tuple(sorted(words)))
    if key in _cache:
        cache_hits += 1
        return _cache[key]

    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        dp[i] = any(dp[j] and s[j:i] in word_set for j in range(i))
    _cache[key] = dp[-1]
    return dp[-1]

print("First call :", can_segment("leetcode", ["leet", "code"]))
print("Second call:", can_segment("leetcode", ["leet", "code"]))
print("Cache hits :", cache_hits)
assert can_segment("leetcode", ["leet","code"]) is True
assert can_segment("leetcode", ["leet","code"]) is True
print("All test cases passed!")
```

Output

```
First call : True
Second call: True
Cache hits : 1
All test cases passed!
```

Question 10: Memory-Constrained Word Break on an Embedded Device

A low-memory embedded device needs to validate short command strings against a small fixed vocabulary of codewords, and must minimize memory used by the segmentation check.

### Approach

Use the boolean word-break DP but discard the parent-pointer reconstruction array entirely, since the device only needs a yes/no validity signal and never needs to recover the actual split.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Acceptable since command strings are short on this device. |
| Space | O(n) | The boolean array alone is kept — the minimum possible for this DP formulation. |

### Test Cases

```
assert can_segment("leetcode", ["leet","code"]) == True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) == False
print('All test cases passed!')
```

main.py

```
def can_segment(s, words):
    word_set = set(words)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[-1]

print("leetcode  ->", can_segment("leetcode", ["leet", "code"]))
print("catsandog ->", can_segment("catsandog", ["cats", "dog", "sand", "and", "cat"]))
assert can_segment("leetcode", ["leet","code"]) is True
assert can_segment("catsandog", ["cats","dog","sand","and","cat"]) is False
print("All test cases passed!")
```

Output

```
leetcode  -> True
catsandog -> False
All test cases passed!
```

## 3. Word Wrap Problem

Question 1: Justifying Text for a Print Newsletter Column

A newsletter layout tool must arrange a sequence of words into lines of a fixed column width, minimizing the total "raggedness" cost, defined as the sum of squared extra spaces at the end of each line (except the last).

### Approach

Define cost(i, j) as the squared extra-space penalty for putting words i..j on one line, or infinity if they don't fit within the width. Build a DP where dp[i] is the minimum total cost for words i..n, choosing among all j where dp[i] = min over valid j of cost(i, j) + dp[j+1].

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Computing cost(i, j) for all pairs and taking a min over each i costs $O(n^2)$ overall. |
| Space | O(n) | The dp array is linear; costs can be computed on the fly with a running prefix sum of word lengths. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert min_wrap_cost([3], 6) == 0  # single word, no penalty on the last line
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

words = [3, 2, 2, 5]
print("Minimum wrap cost =", min_wrap_cost(words, 6))
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert min_wrap_cost([3], 6) == 0
print("All test cases passed!")
```

Output

```
Minimum wrap cost = 10
All test cases passed!
```

Question 2: Chat Bubble Line Wrapping for a Mobile App

A messaging app wants to wrap a long message into chat-bubble lines of limited width, avoiding lines with excessive leftover space that make bubbles look uneven.

### Approach

Model each word's length as in the classic word wrap DP, computing minimum total squared extra-space cost across all valid line breaks, then reconstruct the actual line assignments by tracking the chosen split at each dp[i].

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The cost table and dp fill both cost $O(n^2)$ in the number of words. |
| Space | $O(n^2)$ | Reconstructing line breaks requires keeping a full choice table; O(n) suffices if only the cost is needed. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
lines = wrap_lines([3, 2, 2, 5], 6)
assert sum(len(line) for line in lines) == 4  # all 4 words placed
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

lines = wrap_lines([3, 2, 2, 5], 6)
print("Minimum wrap cost =", min_wrap_cost([3, 2, 2, 5], 6))
print("Optimal lines =", lines)
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert sum(len(line) for line in lines) == 4
print("All test cases passed!")
```

Output

```
Minimum wrap cost = 10
Optimal lines = [[3], [2, 2], [5]]
All test cases passed!
```

Question 3: TeX-Style Badness Minimization for Book Typesetting

A typesetting engine, inspired by TeX's line-breaking algorithm, wants to minimize a "badness" cost across an entire paragraph rather than greedily filling each line, since greedy wrapping can leave the last line badly ragged.

### Approach

Use the global DP formulation rather than a greedy line-fill approach: dp[i] considers every possible line ending j and picks the split minimizing total cost over the whole paragraph, not just the current line, exactly as in classic word wrap.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Every possible line ending is considered for every starting index. |
| Space | O(n) | The dp array alone suffices, with cost(i, j) computed in O(1) using a running prefix sum. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert min_wrap_cost([3, 2, 2, 5], 6) <= greedy_wrap_cost([3, 2, 2, 5], 6)
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def greedy_wrap_cost(words, width):
    lines, current, used = [], [], 0
    for w in words:
        needed = w if not current else w + 1
        if used + needed <= width:
            current.append(w)
            used += needed
        else:
            lines.append(current)
            current, used = [w], w
    if current:
        lines.append(current)
    cost = 0
    for line in lines[:-1]:
        length = sum(line) + len(line) - 1
        cost += (width - length) ** 2
    return cost

words, width = [3, 2, 2, 5], 6
dp_cost = min_wrap_cost(words, width)
greedy_cost = greedy_wrap_cost(words, width)
print("DP cost     =", dp_cost)
print("Greedy cost =", greedy_cost)
assert dp_cost == 10
assert dp_cost <= greedy_cost
print("All test cases passed!")
```

Output

```
DP cost     = 10
Greedy cost = 16
All test cases passed!
```

Question 4: Wrapping Invoice Line-Item Descriptions

An e-invoice generator must fit item descriptions into a fixed-width printed column, minimizing wasted trailing space so the invoice looks professionally formatted.

### Approach

Treat each word in the description as an element in the classic word wrap DP, with the column width as the line-width constraint, and use the same squared-cost minimization to choose line breaks.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Typical item descriptions are short, so the quadratic cost is negligible in absolute terms. |
| Space | $O(n)$ | Invoice descriptions are short enough that the full $O(n^2)$ cost table isn't needed at once. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

main.py

```
def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

lines = wrap_lines([3, 2, 2, 5], 6)
print("Invoice lines =", lines)
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
assert sum(len(line) for line in lines) == 4
print("All test cases passed!")
```

Output

```
Invoice lines = [[3], [2, 2], [5]]
All test cases passed!
```

Question 5: Subtitle Wrapping for Video Captions

A video captioning tool must wrap a spoken sentence into subtitle lines under a fixed character limit per line, minimizing uneven line lengths so captions read comfortably.

### Approach

Apply the standard word wrap DP using the subtitle character-width as the line limit, minimizing the sum of squared leftover space per line, then reconstruct the actual subtitle lines from the chosen breakpoints.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The DP is run once per sentence, over the words in that sentence. |
| Space | O(n) | The dp array and a parent-pointer array used to reconstruct line boundaries are both linear. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

main.py

```
def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

lines = wrap_lines([3, 2, 2, 5], 6)
print("Subtitle lines =", lines)
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
assert sum(len(line) for line in lines) == 4
print("All test cases passed!")
```

Output

```
Subtitle lines = [[3], [2, 2], [5]]
All test cases passed!
```

Question 6: CLI Help-Text Wrapping for a Given Terminal Width

A command-line tool's --help text must wrap descriptive text to fit whatever terminal width is detected at runtime, minimizing ragged line endings for readability.

### Approach

Feed the detected terminal width as M into the classic word wrap DP over the help text's words, recomputing the optimal wrap whenever the terminal is resized.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Each wrap recomputation costs $O(n^2)$ in the number of words in the help text. |
| Space | O(n) | Help text is typically short enough that recomputing from scratch on resize is cheap. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

terminal_width = 6
lines = wrap_lines([3, 2, 2, 5], terminal_width)
print("Terminal width =", terminal_width)
print("Wrapped help text =", lines)
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
print("All test cases passed!")
```

Output

```
Terminal width = 6
Wrapped help text = [[3], [2, 2], [5]]
All test cases passed!
```

Question 7: Balanced Card-UI Reply Formatting for a Chatbot

A chatbot's card-style UI wants replies wrapped into a small, visually balanced number of lines rather than many short ragged ones, penalizing lines that end with a lot of unused space.

### Approach

Use the word wrap DP's squared-cost penalty, which naturally discourages very short trailing lines, since a small amount of leftover space is squared to a small penalty while very short lines incur a large squared penalty, producing visually balanced card text.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The same DP formulation is applied to each reply's word sequence. |
| Space | O(n) | The dp array is linear in the number of words in the reply. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

lines = wrap_lines([3, 2, 2, 5], 6)
print("Balanced card lines =", lines)
print("Raggedness cost =", min_wrap_cost([3, 2, 2, 5], 6))
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
print("All test cases passed!")
```

Output

```
Balanced card lines = [[3], [2, 2], [5]]
Raggedness cost = 10
All test cases passed!
```

## Question 8: Reflow Minimization in a Resizable UI Panel

A resizable text panel wants to minimize how badly text reflows as the panel width changes, by recomputing the optimal word wrap whenever the width crosses a threshold that would otherwise force an ugly layout.

### Approach

Recompute the classic word wrap DP each time the panel width M changes meaningfully, comparing the new optimal cost to the previous layout's cost under the new width to decide whether a re-wrap is worth the visual disruption.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Each recomputation triggered by a width change costs $O(n^2)$ in the number of words. |
| Space | O(n) | The dp array for the new width is kept; the previous layout's table is discarded once a decision is made. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in wrap_lines([3, 2, 2, 5], 6))
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

def reflow(words, new_width):
    return min_wrap_cost(words, new_width), wrap_lines(words, new_width)

cost, lines = reflow([3, 2, 2, 5], 6)
print("New width =", 6)
print("New cost  =", cost)
print("New lines =", lines)
assert cost == 10
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
print("All test cases passed!")
```

Output

```
New width = 6
New cost  = 10
New lines = [[3], [2, 2], [5]]
All test cases passed!
```

## Question 9: Comparing DP Word Wrap to a Greedy Baseline

A course exercise asks students to empirically show that the DP-based word wrap never produces a worse (higher-cost) layout than a simple greedy "fill each line as full as possible" baseline.

### Approach

Implement both the DP word wrap (from Question 1) and a greedy line-filler that packs words onto the current line until the next word wouldn't fit, then compare total costs on the same input to confirm the DP result is always less than or equal to the greedy result.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ / $O(n)$ | The DP version costs $O(n^2)$; the greedy baseline runs in a single $O(n)$ pass. |
| Space | $O(n)$ | Both approaches use dp or line-tracking structures linear in the number of words. |

### Test Cases

```
assert min_wrap_cost([3, 2, 2, 5], 6) <= greedy_wrap_cost([3, 2, 2, 5], 6)
assert min_wrap_cost([3, 2, 2, 5], 6) == 10
print('All test cases passed!')
```

main.py

```
def min_wrap_cost(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            dp[i] = min(dp[i], penalty + dp[j + 1])
    return dp[0]

def greedy_wrap_cost(words, width):
    lines, current, used = [], [], 0
    for w in words:
        needed = w if not current else w + 1
        if used + needed <= width:
            current.append(w)
            used += needed
        else:
            lines.append(current)
            current, used = [w], w
    if current:
        lines.append(current)
    cost = 0
    for line in lines[:-1]:
        length = sum(line) + len(line) - 1
        cost += (width - length) ** 2
    return cost

words, width = [3, 2, 2, 5], 6
dp_cost = min_wrap_cost(words, width)
greedy_cost = greedy_wrap_cost(words, width)
print("DP cost     =", dp_cost)
print("Greedy cost =", greedy_cost)
print("DP is better/equal:", dp_cost <= greedy_cost)
assert dp_cost <= greedy_cost
assert dp_cost == 10
print("All test cases passed!")
```

Output

```
DP cost     = 10
Greedy cost = 16
DP is better/equal: True
All test cases passed!
```

Question 10: Reconstructing Line Breaks for a Long Document

A document renderer needs not just the minimum wrap cost but the actual list of line breaks for a long word sequence, so it can lay out the rendered page.

### Approach

Extend the word wrap DP to record, at each index i, which split point j achieved the minimum cost; after filling the dp array, walk the recorded choices forward from index 0 to reconstruct the full list of lines.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | The DP fill costs $O(n^2)$; the reconstruction walk over the recorded choices adds only O(n). |
| Space | O(n) | Both the dp array and the parent-pointer array used in reconstruction are linear in size. |

## Test Cases

```
lines = wrap_lines([3, 2, 2, 5], 6)
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
assert sum(len(line) for line in lines) == 4
print('All test cases passed!')
```

**main.py**

```python
def wrap_lines(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    next_break = [-1] * n
    dp[n] = 0
    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += words[j] + (1 if j > i else 0)
            if used > width:
                break
            penalty = 0 if j == n - 1 else (width - used) ** 2
            cost = penalty + dp[j + 1]
            if cost < dp[i]:
                dp[i], next_break[i] = cost, j + 1
    lines, i = [], 0
    while i < n:
        j = next_break[i]
        lines.append(words[i:j])
        i = j
    return lines

lines = wrap_lines([3, 2, 2, 5], 6)
print("Reconstructed lines =", lines)
assert all(sum(line) + len(line) - 1 <= 6 for line in lines)
assert sum(len(line) for line in lines) == 4
print("All test cases passed!")
```

**Output**

```
Reconstructed lines = [[3], [2, 2], [5]]
All test cases passed!
```